Prosjekt 1 – TMA4320

Andrea Larssen Aasvik[a], Mari Dybdahl[a], Isabella Juliana Mayorca Johansson[a]

[a] *Institutt for matematiske fag, Norges Teknisk-Naturvitenskapelige Universitet, N-7491 Trondheim, Norway.*

Refleksjonsoppgaver, generering av syntetiske data

Oppgave 3.1

For å finne formler for elementene i den diskretiserte matrisen $\mathbf{A}$ og vektoren $\mathbf{b}^k$ i likningen

$$\mathbf{A}\mathbf{T}^{k+1} = \mathbf{b}^k \tag{1}$$

tar vi systematisk for oss de ulike typene punkter i det diskretiserte romlige domenet Ω; indre punkter, randpunkter og hjørnepunkter. *Indre punkter:* Punkter som ikke ligger på randen til Ω, og dermed ikke krever at vi tar hensyn til randbetingelser. I oppgaveteksten brukes differansemetoden *implicit Euler* for å numerisk løse varmelikningen

$$\frac{\partial T(x,y,t)}{\partial t} = \alpha \Delta T(x,y,t) + q(x,y), \quad (x,y,t) \in \Omega \times [0, t_{\max}], \tag{2}$$

som gir likningen

$$\frac{T_{i,j}^{k+1} - T_{i,j}^{k}}{\Delta t} = \alpha \left(\frac{T_{i+1,j}^{k+1} - 2T_{i,j}^{k+1} + T_{i-1,j}^{k+1}}{(\Delta x)^2} + \frac{T_{i,j+1}^{k+1} - 2T_{i,j}^{k+1} + T_{i,j-1}^{k+1}}{(\Delta y)^2} \right) + q(x_i, y_j) \tag{3}$$

etter å ha satt inn approksimasjoner for de deriverte. Vi definerer videre

$$r_x = \frac{\alpha \Delta t}{(\Delta x)^2}, \quad r_y = \frac{\alpha \Delta t}{(\Delta y)^2}.$$

Etter å ha samlet alle T^{k+1}-ledd i (3) på venstre side, får vi da en likning som er oppfylt av alle indre punkter i Ω:

$$(1 + 2r_x + 2r_y)T_{i,j}^{k+1} - r_x(T_{i+1,j}^{k+1} + T_{i-1,j}^{k+1}) - r_y(T_{i,j+1}^{k+1} + T_{i,j-1}^{k+1}) = T_{i,j}^{k} + \Delta t q(x_i, y_j) \tag{4}$$

Randpunkter: Punktene som ligger på randen, $\partial\Omega$, krever at vi tar hensyn til Robin randbetingelsen

$$-k\nabla T(x,y,t) \cdot \mathbf{n} = h(T(x,y,t) - T_{\text{out}}), \quad (x,y,t) \in \partial\Omega \times [0, t_{\max}]. \tag{5}$$

For venstre rand ($x = 0$), er den utadrettede enhetsnormalvektoren $\mathbf{n} = (-1, 0)$. I henhold til oppgaveteksten bruker vi en *ficticious node*, $T_{-1,j}^{k+1}$, for å approksimere den deriverte på randen, som opptrer i (5). Innsatt i (5) får vi

$$k\frac{T_{0,j}^{k+1} - T_{-1,j}^{k+1}}{\Delta x} = h(T_{0,j}^{k+1} - T_{\text{out}})$$

$$T_{-1,j}^{k+1} = T_{0,j}^{k+1} - \frac{h\Delta x}{k}(T_{0,j}^{k+1} - T_{\text{out}})$$

Satt inn i (4) ved $i = 0$ gir for et punkt på venstre rand

$$(1+r_x+2r_y+\frac{r_x h\Delta x}{k})T_{0,j}^{k+1} - r_x T_{1,j}^{k+1} - r_y(T_{0,j+1}^{k+1}+T_{0,j-1}^{k+1}) = T_{0,j}^{k} + \frac{r_x h\Delta x}{k}T_{\text{out}} + \Delta t q(x_0, y_j) \tag{6}$$

Samme fremgangsmåte gjentas for de resterende randene; høyre rand ved $(x = N_x - 1)$, nedre rand ved $(y = 0)$ og øvre rand ved $(y = N_y - 1)$.

Hjørnepunkter: Punkter som ligger på to rander samtidig. Her bruker vi derfor to Robin randbetingelser for hvert punkt. For $(x = 0, y = 0)$ blir bidraget fra venstre rand, der $\mathbf{n} = (-1, 0)$ og ficticious node er $T_{-1,0}^{k+1}$,

$$T_{-1,0}^{k+1} = T_{0,0}^{k+1} - \frac{h\Delta x}{k}(T_{0,0}^{k+1} - T_{\text{out}}).$$

Fra nedre rand, der $\mathbf{n} = (0, -1)$ og ficticious node er $T_{0,-1}^{k+1}$, blir bidraget:

$$T_{0,-1}^{k+1} = T_{0,0}^{k+1} - \frac{h\Delta y}{k}(T_{0,0}^{k+1} - T_{\text{out}}).$$

Satt inn i (4) ved $i = 0$, $j = 0$, får vi for dette hjørnepunktet likningen

$$(1+r_x+r_y+\frac{r_x h\Delta x}{k}+\frac{r_y h\Delta y}{k})T_{0,0}^{k+1} - r_x T_{1,0}^{k+1} - r_y T_{0,1}^{k+1} = T_{0,0}^{k} + \frac{h}{k}(r_x\Delta x + r_y\Delta y)T_{\text{out}} + \Delta t q(x_0, y_0). \tag{7}$$

Samme fremgangsmåte brukes for å finne likningene som hører til de resterende hjørnene.

Matrise A: Vi har funnet en likning for hver av de $N_x \cdot N_y$ verdiene i vektoren $\mathbf{T}^{k+1}$ i likning (1). Det gir en kvadratisk $(N_x \cdot N_y) \times (N_x \cdot N_y)$ matrise $\mathbf{A}$, der hver rad i matrisen representerer en likning tilhørende et punkt i det diskretiserte domenet Ω. Sammenhengen mellom radnummer i $\mathbf{A}$ og plassering av tilhørende punkt i Ω er den samme som for indekseringen til $\mathbf{T}^{k+1}$. For eksempel, punkt $(i, j) = (0, 0)$ i Ω svarer til indeks p $= i \cdot N_y + j = 0 \cdot N_y + 0 = 0$ i $\mathbf{T}^{k+1}$, og dermed til første rad i $\mathbf{A}$. Det gir at likning (7) som vi utledet over er representert av første rad i $\mathbf{A}$.

Elementene i $\mathbf{A}$ kan beregnes utifra følgende mønster:

Langs hoveddiagonalen til $\mathbf{A}$ er elementene summen av koeffisientene foran $T_{\text{i,j}}^{k+1}$-leddene i likningen som den respektive raden representerer. Likning (7) gir for eksempel at for første rad i $\mathbf{A}$ er diagonalelementet $1 + r_x + r_y + \frac{r_x h\Delta x}{k} + \frac{r_y h\Delta y}{k}$. Sammenhengen mellom et element med en gitt plassering langs diagonalen til $\mathbf{A}$ og plasseringen til det tilhørende punktet i Ω er som følger:

1. De første N_y elementene langs diagonalen hører til venstre rand i Ω. (Merk at elementene 1 og N_y hører til hjørnepunkter.)
2. Element $N_y + 1$ hører til nedre rand.
3. Element $N_y + 2$ til og med element $2N_y - 1$ er indre punkter.
4. Element $2N_y$ hører til øvre rand.

5. Steg 2, 3 og 4 gjentas til hvert av stegene har opptrådt $N_x - 2$ ganger.
6. Etter det $(N_x - 1) \cdot N_y$ diagonalelementet, hører de siste N_y diagonalelementene til høyre rand i Ω. (Merk at elementene $(N_x - 1) \cdot (N_y) + 1$ og $N_x \cdot N_y$ hører til hjørnepunkter.)

Om vi tenker på $\mathbf{A}$ som en matrise videre delt inn i mindre $N_y \times N_y$ matriser, vil vi få en hoveddiagonal i $\mathbf{A}$ bestående av N_x slike matriser. Elementene langs super- og subdiagonalen inne i disse hoveddiagonal-matrisene vil være lik $-r_y$. $N_x \times N_x$-matrisene langs hoveddiagonal-matrisene, altså super- og subdiagonal-matrisene, vil ha egne diagonaler der elementene er lik $-r_x$. For første rad i $\mathbf{A}$ kan man se fra likning (7): koeffisienten foran leddet $T_{0,1}^{k+1}$(representerer element i rad 1, kolonne 2 i $\mathbf{A}$) er lik $-r_y$, og at koeffisienten foran $T_{1,0}^{k+1}$ (representerer element i rad 1, kolonne $1+N_y$ i $\mathbf{A}$) er lik $-r_x$.

Ellers er elementene i $\mathbf{A}$ lik 0.

Vektor b^k: Beskrivelsen for rekkefølgen til elementene langs hoveddiagonalen til $\mathbf{A}$ gjelder også elementene i $\mathbf{b}^k$, bare at for $\mathbf{b}^k$ er det høyresiden av hver likning som blir elementet ved den aktuelle indeksen. For eksempel hører det første elementet i $\mathbf{b}^k$ til hjørnepunktet $(x = 0, y = 0)$ i Ω. Fra likning (7) får vi som første element i $\mathbf{b}^k$

$$T_{0,0}^k + \frac{h}{k}(r_x \Delta x + r_y \Delta y) T_{\text{out}} + \Delta t q(x_0, y_0).$$

Resterende elementer beregnes på samme måte: finne ut hvilket punkt i Ω elementet ved denne indeksen korresponderer til, bruke rett likningstype (indre punkt, randpunkt eller hjørnepunkt), og sette inn for riktig i og j. Høyresiden i likningen er da det aktuelle elementet i $\mathbf{b}^k$.

For å forstå hvordan $\mathbf{A}$ og $\mathbf{b}^k$ så ut, brukte vi i starten en del tid på å forstå implementeringen av funksjonene henholdsvis $build_matrix()$ og $build_rhs()$ i src/project/fdm.py-filen, og brukte disse i stor grad som utgangspunkt for å kunne beskrive verdi og plassering til elementene i matrisen og vektoren. Fordi disse funksjonene ble utdelt med oppgaven anså vi det ikke som nødvendig å legge ved skjermbilder av dem for å kunne henvise til i denne oppgaven.

Oppgave 3.4

En modell som den vi beskriver i dette prosjektet kan redusere energiforbruket under oppvarmingen av hus på flere måter. For det første, ved å simulere med ulike verdier av parameterene k (veggenes termiske ledningsevne) og h (varmeovergangskoeffisienten) i Robin randbetingelsene (likning (5)) kan man kartlegge igjennom for eksempel hvilke vegger det er størst varmetap. På den måten er det mulig å iverksette målrettet etterisolering av huset, og slik redusere varmetapet og energiforbruket. I det store bildet går det også å tenke at å unngå unødvendig, overflødig isolasjon vil være energibesparende på lang sikt, om ikke direkte ved oppvarmingen av huset. Ved å variere definisjonen av funksjonen q i likning (2), går det an å finne ut hvor det er mest mulig gunstig å plassere varmekilder. Videre skaper en slik modell muligheter for å ha varmekilder som justerer effekten sin automatisk med hensyn på utendørs temperatur (parameteren T_{out} i likning (5)) og ønsket temperatur innendørs, og dermed unngå overflødig oppvarming. En mulig idé er å gi varmekildene tilgang til værmelding, slik at de for eksempel raskere kan justere ned effekten i tilfelle varmere temperaturer utendørs. Siden modellen er tidsavhengig,

kan den også brukes til å undersøke om for eksempel en langsom oppvarming med lavere effekt hos varmekilden eller en hurtig oppvarming med høyere effekt vil ha lavest energiforbruk. Ved å variere antall sensorer og plasseringen av dem i modellen, kan den også brukes til å optimalisere plasseringen av sensorene, slik at gjengivelsen av den faktiske temperaturfordelingen i rommet blir så virkelighetsnær som mulig.

Oppgave 3.5

Den største forskjellen mellom implisitte og eksplisitte metoder er at ved implisitte metoder løser man ligningssystem som inneholder kjente verdier fra både nåværende tidssteg og fremtidige verdier fra neste tidssteg, mens ved eksplisitte metoder bruker man kun kjente verdier fra et tidssteg for å beregne neste. Altså vil man ved bruk av implisitte metoder få store ligningssett som må løses ved hvert tidssteg, noe som gjør at de kan være tregere enn eksplisitte metoder. Imidlertid er implisitte metoder mer stabile enn eksplisitte metoder, og tillater å ta større tidssteg uten at løsningen divergerer.[1] [2]

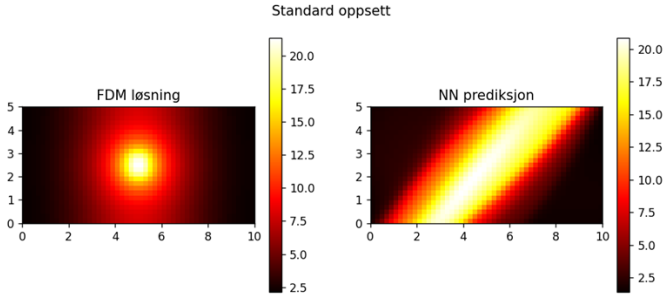
Heldigvis fins det metoder som kan gjøre at beregningen av disse ligningssettene går raskere. I oppgave 3.1 og 3.2 fikk vi en symmetrisk, «sparse» matrise. Dette åpner for bruk av f.eks. «conjugate gradient»-metoden, dersom matrisen også er positiv bestemt. CG-metoden går ut på at den søker etter løsninger langs «konjugerte retninger» i stedet for å bare f.eks. gå langs retningen hvor residualen $r_k = \mathbf{b} - \mathbf{A}\mathbf{T}^k$ minker. Med dette unngår man å «ødelegge» minimeringen man allerede har gjort i forrige steg, noe som gir raskere konvergens mot løsningen. Ved bruk av CG-metoden er det ofte nødvendig med «preconditioning» for å redusere antall iterasjoner som må til før løsningen konvergerer. Dette gjøres ved å multiplisere ligningssystemet med en «precondition»-matrise, $\mathbf{M}^{-1}$, slik at man får et system som er enklere å løse. $\mathbf{M}$ er typisk en matrise som ligner på $\mathbf{A}$, f.eks. «Jacobi preconditioner» som er lik den diagonaliserte matrisen diag($\mathbf{A}$). En annen teknikk for å redusere kjøretiden er parallellisering av løseren. Ved parallellisering deler man opp beregningene i løseren slik at flere prosessorer kan jobbe samtidig. Dette fungerer spesielt godt på vårt problem da elementene i vår matrise kun er avhengig av sine nabo-elementer.

Oppgave 3.6

For å estimere de fysiske parameterne α, k og h kan vi sette det opp som et optimeringsproblem. Ettersom vi har både sensordata og en numerisk løser, har vi mulighet til å finne T på to ulike måter. Det vi nå ønsker er å velge parametrene slik at forskjellen mellom temperaturen målt av sensorene, T_{obs}, og temperaturen vi kommer frem til numerisk, T_{num}, blir så liten som mulig, f.eks. ved bruk av minste kvadraters metode. Altså kan man få en tapsfunksjon på denne formen:

$$\mathcal{L}(\alpha, k, h) = \sum_{i=1}^{N} \left(T_{\text{num}}(x_i, y_i, t_i; \alpha, k, h) - T_{\text{obs}}(x_i, y_i, t_i)\right)^2 \tag{8}$$

hvor man summerer over et utvalg av sensordata. Det er ulike måter å minimere $\mathcal{L}$ på, men en metode kan være å bruke «Gradient descent» som iterativt justerer α, k og h slik at feilen $\mathcal{L}$ konvergerer. Det er også viktig å passe på at α, k og h holdes positive, f.eks. ved å bruke log-verdiene underveis.



Figur 1: Standard oppsett av FDM løsning og NN prediksjon

Refleksjonsoppgaver, nevrale nettverk

Oppgave 4.5

For å undersøke om et standard nevralt nettverk kan lære temperaturfeltet utelukkende fra sensordata, ble prediksjonene fra nettverket sammenlignet med en referanseløsning beregnet med Finite Difference Method (FDM). Sammenligningen ble gjennomført både visuelt gjennom temperaturplott og kvantitativt ved hjelp av Mean Squared Error (MSE) og relativ feil.

0.1. Sammenligning av temperaturfelt

Som vist i 1 fremstår temperaturfeltet fra FDM som rundt og tilnærmet symmetrisk rundt varmekilden, i samsvar med den forventede løsningen av varmeligningen. Prediksjonen fra det nevrale nettverket viser derimot en tydelig diagonal struktur. Dette indikerer at nettverket ikke klarer å gjenskape den faktiske fysiske oppførselen når treningen baseres på et begrenset antall sensormålinger. Metodene skiller seg også prinsipielt. FDM beregner temperaturen direkte fra den diskretiserte varmeligningen i hele domenet, mens det nevrale nettverket må lære en funksjon $T(x, y, t)$ kun fra målepunkter. Uten eksplisitt informasjon om ligningen eller grensebetingelsene vil nettverket i hovedsak interpolere mellom sensorene, noe som kan forklare den urealistiske formen på prediksjonen.

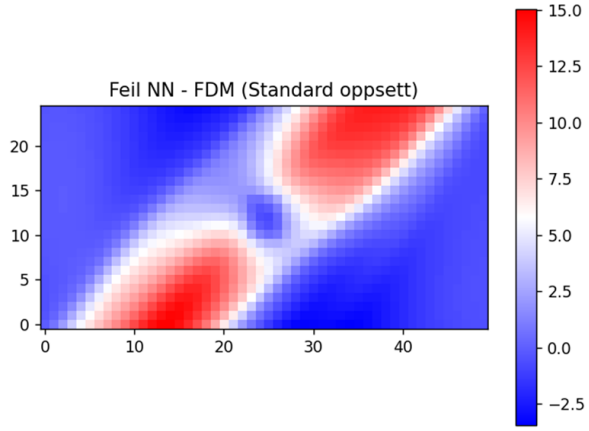
Feilfeltet i **??** viser at de største avvikene ligger langs den diagonale strukturen nettverket har lært, mens FDM-løsningen er mer konsentrert rundt varmekilden der temperaturgradienten faktisk er størst.

0.2. Kvantitativ evaluering

De numeriske resultatene er oppsummert i 1. Nettverket gir høy feil i alle testede tilfeller. En relativ feil over 100 % innebærer at prediksjonen i gjennomsnitt avviker mer fra FDM enn selve størrelsen på løsningen, noe som tydelig viser at NN alene ikke er tilstrekkelig for å rekonstruere temperaturfeltet på en pålitelig måte.

0.3. Effekt av antall sensorer

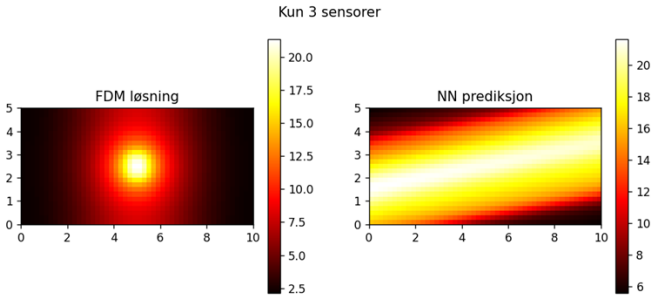
Antall sensorer har betydelig innvirkning på resultatene. Med kun tre sensorer, vist i 3, blir prediksjonen sterkt forenklet og tilnærmet lineær. MSE øker da til 74.22, over



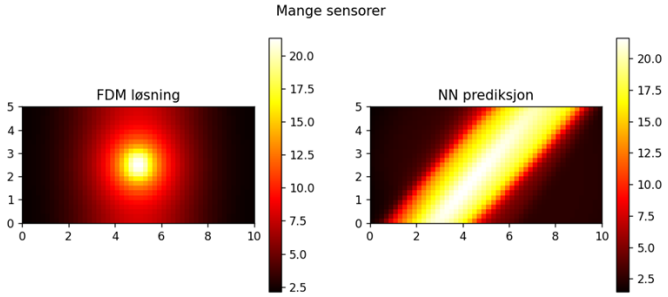
Figur 2: Feilfelt for NN ved standard oppsett

Ulike tilfeller	MSE	Relativ feil
Standard	21.26	104.9 %
Kun 3 sensorer	74.22	195.9 %
Mange sensorer	24.03	111.5 %
Høy støy	34.46	133.5 %

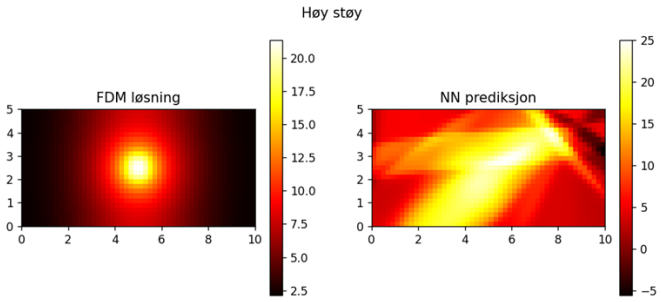
Tabell 1: MSE og relativ feil for ulike scenarioer



Figur 3: FDM løsning og NN prediksjon for 3 sensorer



Figur 4: FDM løsning og NN prediksjon for flere sensorer



Figur 5: FDM løsning og NN prediksjon ved høy støy

tre ganger høyere enn i standardoppsettet. Dette er forventet ettersom nettverket mottar svært begrenset informasjon om den romlige temperaturvariasjonen.

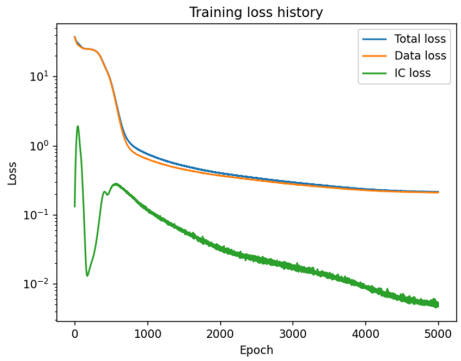
Når antallet sensorer økes, forbedres resultatet som vist i 4. MSE reduseres til 24.03, og feltet får flere detaljer. Prediksjonen beholder likevel en diagonal struktur som ikke finnes i FDM-løsningen, noe som viser at økt datamengde alene ikke er tilstrekkelig for å lære den korrekte fysiske formen.

0.4. Effekt av støy

Støynivået påvirker læringen i stor grad. Ved høy støy øker MSE til 34.46, og prediksjonen i 5fremstår ujevn og lite fysisk realistisk. Dette illustrerer en grunnleggende svakhet ved rene datadrevne modeller: nettverket har ingen mekanisme for å skille mellom reelle temperaturvariasjoner og målefeil, og lærer derfor også støyen i dataene.

0.5. Effekt av epoker og Adam-parametere

Tapskurven i 6 viser stabil konvergens over epokene, noe som indikerer at Adam-algoritmen og valgt læringsrate fungerer tilfredsstillende. Feilen flater imidlertid ut på et relativt høyt nivå. Dette tyder på at hovedbegrensningen ikke ligger i optimaliseringen, men i selve modelltilnærmingen, ettersom nettverket mangler informasjon om varmeligningen som FDM eksplisitt tar hensyn til.



Figur 6: Tapskurve

Resultatene viser at et standard nevralt nettverk ikke klarer å gjenskape temperaturfeltet tilfredsstillende når treningen kun baseres på sensordata. Prediksjonene avviker systematisk fra FDM-løsningen, og både MSE og relativ feil forblir høye i alle scenarioer. Antall sensorer og støynivå har stor betydning: få sensorer gir sterkt forenklede felt, mens høy støy fører til ustabile prediksjoner. Dette understreker begrensningene ved rene datadrevne modeller og gir et tydelig faglig grunnlag for å ta i bruk fysikk-informerte metoder i videre arbeid.

Refleksjonsoppgaver, fysikk-informerte nevrale nettverk

Oppgave 5.5

For å evaluere resultatene sammenlignet vi den numeriske løseren (FDM) med resultatene fra NN og PINN. For å gjøre dette skrev vi en kode med utgangspunkt i scripts/run_nn.py og scripts/run_pinn.py, bare at disse ble kjørt flere ganger med ulike sensor-innstillinger for å simulere ulike scenarioer (jfr. oppgave5_5.py i scripts-mappen). Her sammenlignet vi fire ulike scenarioer:

1. Ingen endring i forhold til det som var oppgitt i config.yaml
2. Færre sensorer enn originalt (kun de tre første)
3. Flere sensorer enn originalt (25 sensorer spredt jevnt ut i rommet)
4. Mer støy på sensordataen ($\sigma_e = 5.0$)

For å sammenligne NN og PINN med den numeriske løseren valgte vi å bruke den numeriske løsningen som en fasit, og regnet ut den relative feilen basert på denne etter ligningen under

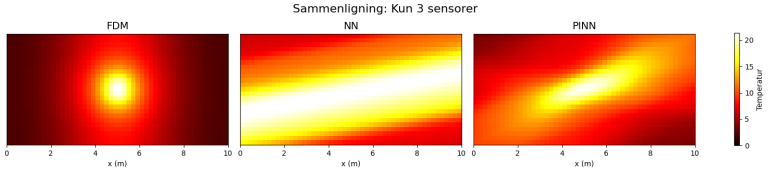
$$\text{Relativ feil} = \frac{\|T_{(\text{PI})\text{NN}} - T_{\text{num}}\|}{\|T_{\text{num}}\|}, \tag{9}$$

hvor $T_{(\text{PI})\text{NN}}$ er matrisen med temperaturer vi får etter å ha trent nettverket, og T_{num} er «fasit»-matrisen regnet ut av den numeriske løseren.

Tabell 2 oppsummerer de relative feilene vi fikk for de ulike scenarioene. Disse verdiene er imidlertid generert fra én kjøring. Ettersom initialiseringen av nettverksvektene og de fysiske parametrene genereres tilfeldig hver gang, vil disse verdiene variere noe fra kjøring

Tabell 2: Relative feil hos NN og PINN ved ulike antall sensorer og støynivå.

Scenario	Antall sensorer	Støynivå (σ_e)	Feil, NN	Feil, PINN
1	9	0.5	111.28 %	15.91 %
2	3	0.5	185.63 %	101.17 %
3	25	0.5	43.50 %	11.73 %
4	9	5.0	118.49 %	60.50 %

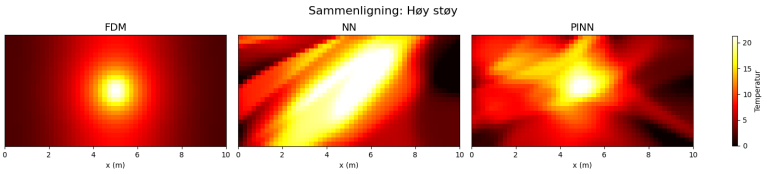


Figur 7: Visualisering av scenario 2: 3 sensorer, $\sigma_e = 0.5$

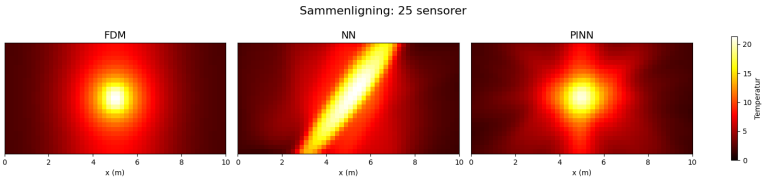
til kjøring, men trendene mellom metodene kan anses som representative. Som vi ser i tabellen fikk vi lavest relativ feil hos både NN og PINN da det var mange sensorer og lavt støynivå. Dette gir mening ettersom et lavt støynivå betyr mer presis sensordata, og mange sensorer gir bedre romslig oppløsning for treningen. Tabellen tydeliggjør også forskjellen mellom NN og PINN. I alle scenarioene ser vi at PINN kommer betydelig nærmere den faktiske løsningen enn NN. NN har relative feil på over 100 % og i beste fall en feil på 43.50 %. En modell som gjetter 0°C overalt vil ha en relativ feil på 100 %, som betyr at NN fungerer veldig dårlig i mange av scenarioene. De store feilene viser at NN ikke klarer å «lære seg» varmeligningen kun basert på punktene alene, og at den kun prøver å tilnærme seg en funksjon som passer til sensordataen. Av denne grunnen blir treningen av NN spesielt sensitiv til antall sensorer og støynivået hos sensorene.

Figur 7 viser temperaturfordelingen i rommet ved siste tidspunkt ($t = 24$ h) i scenario 2. De tre sensorene som er med i dette scenarioet er plassert i midten av rommet (5.0, 2.5) og ved koordinatene (5.0, 5.0) og (5.0, 0.0), altså i midten av øverste og nederste vegg. Om man sammenligner den numeriske løsningen og NN ser man at de er veldig like rundt området der sensoren er plassert, mens utenfor har ikke NN noe data å ta utgangspunkt i. Dermed finner NN en løsning som er ulik den numeriske. PINN viser derimot et bilde som ligner mer på den numeriske løsningen ettersom dette nettverket vet hvordan varme oppfører seg, og dermed har mer informasjon å approksimere løsningen fra. Likevel er det ingen sensorer på venstre og høyre side og dermed blir det fortsatt en stor relativ feil.

Figur 8 viser scenario 4 og demonstrerer hvordan NN stoler mer «blindt» på sensordataen. Her er standardavviket i den gaussiske støyen satt opp fra 0.5 til 5.0 og dermed får man veldig upålitelige målinger. NN har kun disse målingene, i tillegg til intialbetingelsene, å approksimere løsningen fra og finner dermed en løsning som er overtilpasset sensordataen og ikke gir fysisk mening. Igjen er PINN bedre på å approksimere løsningen. Ettersom objektivfunksjonen tar hensyn til sensordata vil PINN også gjøre dette, men den vil ikke overtilpasse dataen i like stor grad da den også prøver å finne en løsning som gir fysisk mening. Varmeligningen er en diffusjonsprosess som naturlig «glatter ut» temperaturforskjeller, og som vi ser i figur 8 finner PINN fortsatt en løsning som i stor



Figur 8: Visualisering av scenario 4: 9 sensorer, $\sigma_e = 5.0$

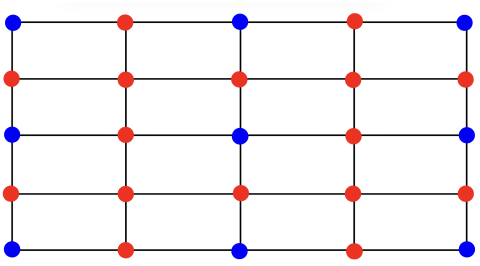


Figur 9: Visualisering av scenario 3: 25 sensorer, $\sigma_e = 0.5$

grad innebærer en varmekilde i midten av rommet og at varmen sprer seg utover. Denne løsningen er også mye glattere enn løsningen fra NN.

Figur 9 viser scenario 3 hvor det er 25 sensorer. De 25 sensorene er plassert i rutenett med 5×5 jevnt fordelte sensorer, som vist i figur 10. Dette er scenarioet hvor både NN og PINN fikk lavest relativ feil. NN finner fortsatt en slags diagonal varmekilde, men den ligner mer på numeriske løsningen sammenlignet med hva NN fant i andre scenarioene. Bildet til PINN ligner veldig på den numeriske løsningen, bare at den har stråler med høyere temperatur som går ut i kryss til der sensorene er plasserte. Ved å studere denne visualiseringen kan man se at PINN finner en for lav temperatur i områdene lenger unna sensorene. Dette kan skyldes at den lærte kildestyrken, P, er alt for lav. Selv om visualiseringen av PINN og den numeriske løsningen ligner, er de lærte parameterene langt unna de sanne verdiene, som vist i tabell 3. Dette tyder på at PINN har funnet en kombinasjon av parameterene som gir et lokalt minimum i tapsfunksjonen, hvor feilene i parameterene kompenserer for hverandre.

Oppgave 5.6 For å måle hvor godt PINN klarer å lære de fysiske parametrene α, k, h og P, regner vi ut den relative feilen mellom de sanne parameterverdiene brukt i den



Figur 10: Sensorene som brukes i treningen av PINN og NN. De blå sensorene er de 9 sensorene som lå i config.yaml fra før, og de røde sensorene er de vi lå til i scenario 3.

Tabell 3: Lærte fysiske parametere vs. sanne verdier for scenario 3.

Parameter	Sann verdi	Lært verdi	Relativ feil
α	0.25	0.0192	92.32 %
k	0.04	0.2951	637.63 %
h	0.00	0.0855	- %
P	15.0	1.5315	89.79 %

Tabell 4: Lærte fysiske parametere vs. sanne verdier ved opprinnelig konfigurasjon.

Parameter	Sann verdi	Lært verdi	Relativ feil
α	0.25	0.02794	88.82 %
k	0.04	0.2255	463.74 %
h	0.00	0.0950	- %
P	15.0	1.8914	87.39 %

numeriske løseren og verdiene PINN har lært. For α blir relativ feil

$$\text{Relativ feil } \alpha = \frac{\|\alpha_{\text{PINN}} - \alpha_{\text{num}}\|}{\|\alpha_{\text{num}}\|}$$

der α_{PINN} er den lærte verdien av α, og α_{num} er den sanne parameterverdien. I tabell 4 kan man lese verdier fra én kjøring av kode fra filen oppgave5_6.py i scripts-mappen. Her er de opprinnelige verdiene oppgitt i filen config.yaml brukt. I eksempelet i seksjon 4.1.1 i [3], blir de relative feilene for de PINN-trente fysiske parameterne λ_1 og λ_2, som inngår i Navier-Stokes-likningene for to dimensjoner, henholdsvis 0.17% 5.70%. I tillegg til å være trent for å tilfreddstille et annet fysisk prinsipp, har også PINN-et i eksempelet blant annet en annen nettverksarkitektur enn vårt. Vi kan med andre ord ikke forvente å få like resultater for vårt PINN. Eksempelet sier likevel noe om hvor godt et PINN *kan* være til å lære seg fysiske parametre. Det er rimelig å si at vårt PINN ikke klarer å lære seg de fysiske parametrene særlig godt (med de opprinnelige oppgitte verdiene i config.yaml).

En mulig forklaring på de store relative feilene for våre fysiske parametre kan vi få ved å se på verdien til tapsfunksjonen $\mathcal{L}_{\text{ph}}$. For den kjøringen av koden som ga verdiene i tabell 4, var $\mathcal{L}_{\text{ph}} = 0.0932$. Og øker vi verdien til λ_{ph} til 100, slik at $\lambda_{\text{ph}} = 100 \cdot \lambda_{\text{data}}$, og nettverket i større grad prioriterer å overholde varmelikningen enn sensordataene (se oppgave 5.7), gir en ny kjøring av koden at $\mathcal{L}_{\text{ph}} = 0.0096$. (Økingen av λ_{ph} ble her gjort ved å endre selve verdien i filen config.yaml). Feilen i parametrene α og P, som er de fysiske parametrene som opptrer i utregningen av $\mathcal{L}_{\text{ph}}$, forblir likevel høye. Det samme gjelder feilene i parametrene k og h, som brukes i $\mathcal{L}_{\text{bc}}$, dersom vi vekter $\mathcal{L}_{\text{bc}}$ tyngre. Nettverket klarer altså å lære seg en løsning for temperaturen som er nokså fysisk konsistent, men der de fysiske parametrene ikke stemmer overens med de sanne. Det er nok en konsekvens av hvordan vårt PINN trenes; det er ikke direkte gale parameterverdier som straffes, men brudd på varmelikningen.

Det kan finnes flere årsaker til at vi har få slike brudd selv med gale parametre. For eksempel kan feil i én parameter kompenseres av feil i en annen. I tillegg opptrer styrken til varmekilden, P, kun i funksjonen $q(x, y)$, som har verdi lik 0 og ikke er avhengig

av P for (x, y) i store deler av det diskretiserte domenet Ω. Dermed kan det hende at betydningen som P sin verdi har for $\mathcal{L}_{\text{ph}}$, blir lav. Støy i sensordata kan også ha vært en årsak; det at modellen forsøke å tilfredsstille fysiske prinsipper samtidig som data som ikke representerer virkeligheten helt riktig, kan gjøre at modellen justerer parameterne feil. Som beskrevet i avsnittet ovenfor opplevde vi likevel at da vi justerte vektene slik at forholdet mellom λ_{ph} og λ_{data} var to størrelsesordener, så ble den relative feilen i parametrene fortsatt stor.

Oppgave 5.7 Vektene λ_{data}, λ_{ic}, λ_{ph} og λ_{bc} avgjør hvor mye de forskjellige komponentene som inngår i objektivfunksjonen $\mathcal{L}$ skal vektlegges. En høy verdi av λ_{data} vil f.eks. tvinge nettverket til å prioritere sensordataene, og motsatt vil en høy λ_{ph} gjøre at nettverket til en større grad overholder varmeligningen.

Ideelt sett ønsker vi å velge store vekter for å minimere alle feilene samtidig, men dette kan gjøre treningen av nettverket ustabilt, og gjør det vanskeligere for å få til å konvergere, da gradienten fra de ulike leddene kan peke i motsatt retning. Dessuten er det heller ikke mulig å finne en perfekt løsning, da sensordataen inneholder støy og dermed ikke vil kunne sammenfalle med den glatte, fysiske løsningen som varmeligningen beskriver.

Det er derfor viktig å reflektere over hvordan man skal velge de ulike vektene. Om man velger for stor λ_{data} kan det resultere i at nettverket overtilpasser sensordataen. Ettersom sensordataen inneholder støy vil dette føre til en løsning som ikke gir mening fysisk. Om man vet at det er mye støy i sensordataen vil det derfor være smart å velge en lavere vekt for objektivfunksjonen til sensordataen. Hvis man derimot velger en høy λ_{ph} slik at nettverket tilpasser seg varmeligningen, kan man risikere at løsningen ignorerer målingene dersom de fysiske parametrene (α, k, h og P) ikke er estimert enda. Det er uansett viktig å passe på at én vekt ikke blir for stor, da dette kan føre til at tilhørende ledd dominerer og at de andre objektivfunksjonene blir ignorert.

En fremgangsmåte når man skal velge vekter kan være å først sette alle vektene lik 1, kjøre et par epoker, og se hvordan de forskjellige objektivfunksjonene, $\mathcal{L}_{\text{data}}$, $\mathcal{L}_{\text{ic}}$, $\mathcal{L}_{\text{ph}}$ og $\mathcal{L}_{\text{bc}}$, utvikler seg. Om f.eks. $\mathcal{L}_{\text{data}}$ blir mye større enn $\mathcal{L}_{\text{ic}}$ så kan man da justere opp λ_{ic} og justere ned λ_{data} slik at begge leddene bidrar omtrent like mye i total-tapet.

Tabell 5 viser effekten på tapsfunksjonene ved å velge ulike vekter. Vi eksperimenterte med å først variere de ulike vektene hver for seg, og til slutt lagde vi en kombinasjon med $\lambda_{\text{ph}} = \lambda_{\text{bc}} = 25.0$, $\lambda_{\text{ic}} = 10.0$ og $\lambda_{\text{data}} = 1.0$. Den siste kombinasjonen ble valgt basert på resultatene av eksperimentene over, som viste at å øke λ_{ph} og λ_{bc} hadde størst effekt på den relative feilen mellom temperatur-matrisen som ble laget av PINN og matrisen regnet ut numerisk. Et viktig funn er at $\mathcal{L}_{\text{data}}$ ble betydelig høyere her enn ved de andre kombinasjonene, til tross for at den relative feilen var lav. Ettersom sensordataen inneholder støy er det viktig at $\mathcal{L}_{\text{data}}$ ikke blir for lav da dette betyr at PINN kan ha overtilpasset seg støyen, så en høyere $\mathcal{L}_{\text{data}}$ betyr ikke nødvendigvis at approksimeringen ble dårligere.

For å unngå å måtte velge vektere manuelt er det også mulig å benytte adaptive metoder. Dette er algoritmer som justerer vektene underveis i treningen basert på størrelsen til gradientene til de ulike objektivfunksjonene. Dette sikrer en mer stabil trening uten behov for å prøve seg frem.

Oppgave 5.8 Fysikkinformerte nevrale nettverk (PINNs) kombinerer maskinlæring med kjente fysiske lover for å løse differensialligninger. I stedet for å diskretisere problemet slik som i tradisjonelle numeriske metoder, brukes et nevralt nettverk til å repre-

Tabell 5: Effekten av ulike vektinger (λ) på tapsfunksjonene og relativ feil (fra ligning (9)) etter 1000 epoker. For å regne ut disse vektene brukte vi filen oppgave5_7.py i scripts-mappen og varierte vektene inne i config.yaml.

Konfigurasjon	$\mathcal{L}_{\text{ph}}$	$\mathcal{L}_{\text{ic}}$	$\mathcal{L}_{\text{bc}}$	$\mathcal{L}_{\text{data}}$	Relativ feil
Alle $\lambda = 1$	0.438	0.115	0.045	0.576	55.94 %
$\lambda_{\text{ph}} = 10$	0.224	0.229	0.044	1.133	29.37 %
$\lambda_{\text{data}} = 10$	0.885	1.149	0.533	0.565	92.90 %
$\lambda_{\text{ic}} = 10$	0.520	0.006	0.118	0.859	66.67 %
$\lambda_{\text{bc}} = 10$	0.479	0.095	0.004	0.576	51.49 %
$\lambda_{\text{ph, bc}} = 25$, $\lambda_{\text{ic}} = 10$, $\lambda_{\text{data}} = 1$	0.164	0.007	0.008	3.086	19.78 %

sentere løsningen direkte. Nettverket trenes ved å minimere en tapsfunksjon som består av bidrag fra initialbetingelser, randbetingelser og et residualledd som uttrykker selve differensialligningen.

En sentral fordel med PINNs er muligheten til å bruke både fysiske lover og målte data i samme modell. Dette er spesielt nyttig når datagrunnlaget er begrenset eller inneholder støy, fordi modellen kan følge de underliggende fysiske sammenhengene i stedet for å basere seg på ren interpolasjon mellom datapunkter. I arbeidet med temperaturproblemet ble dette tydelig: selv når sensordataene alene ga et lite realistisk felt, tvang fysikkleddet løsningen i en mer fornuftig retning. Metoden er derfor aktuell innen blant annet fluidmekanikk og elektromagnetisme [4].

PINNs kan også være nyttige når de fysiske parameterne i et system er ukjente, slik som α, h, k og Pi dette prosjektet. I slike tilfeller kan klassiske numeriske løsere være vanskelige å bruke direkte, mens et PINN i prinsippet kan lære både temperaturfeltet og parameterne samtidig. Hvor godt dette faktisk fungerer i praksis, virker likevel å avhenge mye av hvordan treningen settes opp.

Tapsfunksjonen: $L(\theta) = L_{ic} + L_{bc} + L_r$

består av ledd som måler avvik fra initial- og randbetingelser samt residualen fra differensialligningen. Gradientene som inngår i residualleddet beregnes ved hjelp av automatisk derivasjon. I implementasjonen ble JAX benyttet, og det ble raskt klart at koden måtte skrives i en mer funksjonell stil for å få utnyttet jit- og vmap-transformasjonene. Uten disse optimaliseringene ble treningen merkbart tregere enn forventet.

Sammenlignet med metoder som FDM og FEM krever ikke PINNs en forhåndsdefinert mesh, noe som gir større fleksibilitet for komplekse geometrier. Samtidig har metoden utfordringer, særlig knyttet til balansering av tapsleddene L_{ic}, L_{bc}, L_r, som ofte opererer på ulike skalaer. I noen av testene førte dette til at modellen prioriterte randbetingelsene på bekostning av selve differensialligningen, noe som ga fysisk tvilsomme løsninger. Forskning har foreslått adaptive vekter for å håndtere dette [4]

En mer grunnleggende utfordring er spektral bias: nevrale nettverk lærer lavfrekvente løsninger raskere enn høyfrekvente detaljer. Dette kan være problematisk for systemer med skarpe fronter eller raske variasjoner, som nettopp er vanlige i varmeledning. Teknikker som Fourier-embedding er foreslått for å dempe effekten [4], men dette ble ikke testet her.

Oppsummert gir PINNs en fleksibel tilnærming til fysikkbasert modellering, men erfaringene fra prosjektet viser at metoden krever en del prøving og feiling før den fungerer godt. Løsningene kan bli fysisk konsistente, men resultatet er følsomt for vekting, arki-

tektur og treningsstrategi, og metoden fremstår derfor mer som et kraftig, men ikke helt modent alternativ til klassiske numeriske metoder.

Oppgave 5.9

Arbeidet med implementeringen av PINNs har vist at metoden kan være et interessant verktøy for å kombinere sensordata med fysikkbasert modellering. Samtidig ble det raskt tydelig at treningen er følsom for flere valg som må tas underveis, særlig når det gjelder sampling av punkter, balansering av tapsledd og utforming av nettverket.

En av de tydeligste utfordringene var vektingen mellom de ulike tapsleddene. I koden ble den totale tapsfunksjonen formulert som

$$L_{\text{total}} = \lambda_{\text{data}} L_{\text{data}} + \lambda_{\text{ic}} L_{\text{ic}} + \lambda_{\text{ph}} L_{\text{ph}} + \lambda_{\text{bc}} L_{\text{bc}}$$

, med faste hyperparametere for hvert ledd. Denne tilnærmingen fungerer i utgangspunktet greit, men under treningen oppsto situasjoner der ett ledd dominerte og styrte løsningen i en retning som ikke alltid virket fysisk rimelig. Et naturlig neste steg vil derfor være å teste adaptive vektingsstrategier som justerer bidragene underveis, slik at modellen i større grad lærer både fra data og fra differensialligningen samtidig.

Også samplingstrategien viste seg å ha mye å si. I den nåværende implementasjonen trekkes treningspunkter tilfeldig i hver epoke. Dette er enkelt å implementere, men det er ikke sikkert at punktene da havner der modellen faktisk gjør størst feil. En mer målrettet tilnærming kunne vært adaptiv sampling, der nye punkter legges i områder med høy residual. Det virker sannsynlig at dette kunne gitt raskere og mer stabil læring, spesielt nær varmekilden der gradientene er størst.

Når det gjelder nettverksarkitekturen, er den som brukes her relativt enkel. Resultatene tyder på at nettverket sliter med å representere mer detaljerte strukturer i temperaturfeltet. En mulig forbedring er å ta i bruk Fourier-features eller andre inputtransformasjoner som gjør det lettere å lære høyfrekvente variasjoner \cite{wang2023}. Dette kan være særlig relevant dersom metoden senere skal brukes på mer kompliserte problemer enn det som er testet her.

Implementasjonen gir også noen interessante muligheter utover ren temperaturprediksjon. I prosjektet ble termisk diffusivitet estimert som en trenbar parameter, og dette kan i prinsippet utvides til å gjelde flere eller mer komplekse materialegenskaper. Det ville for eksempel vært spennende å teste om modellen kan håndtere romlig varierende parametere, noe som er mer realistisk i praktiske ingeniørsystemer.

En idé som dukket opp underveis, er å bruke modellen til å si noe om hvor sensorer bør plasseres. Dersom PINN-et kan identifisere områder med høy usikkerhet, kan dette brukes til å foreslå bedre sensoroppsett i stedet for å plassere dem vilkårlig. Om dette faktisk fungerer i praksis er uklart, men det virker som en naturlig videreføring av arbeidet.

Samlet sett viser prosjektet at PINNs er en fleksibel metode med stort potensial, men også at den krever en del prøving og feiling før resultatene blir gode. Valg av vekting, sampling og arkitektur har større betydning enn forventet, og videre arbeid med adaptive metoder vil trolig være nødvendig for at tilnærmingen skal bli robust nok for reelle ingeniøroppgaver.

Referanser

[1] Kollmannsberger, S; D'Angella, D; Jokeit, M; Herrmann, L; Deep learning in computational mechanics—an introductory course. Stud. Comput. Intell., 977 Springer,

Cham, [2021], ©2021. vi+104 pp

[2] Sauer, T; Numerical Analysis, 2nd Ed, Pearson, 2012.

[3] Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics 378, 686–707.. https://doi.org/10.1016/j.jcp.2018.10.045

[4] Wang, S., Sankaran, S., Wang, H., Perdikaris, P. (2023). An expert's guide to training physics-informed neural networks. arXiv preprint arXiv:2308.08468. https://arxiv.org/abs/2308.08468